

Name : Neil Taison RIGAUD
Student ID : 411221367
Subject : Intro to Image Processing – Assignment #3
Date : 2025/11/01

1- Code Listing and Explanation

```
13
14 import os
15 import numpy as np
16 from PIL import Image
17
18 INPUT_IMAGE = "lena_gray.png"
19 OUTPUT_DIR = "filtered_images"
20 NOISY_PIXEL_LOW = 500
21 NOISY_PIXEL_HIGH = 10000
22 PEPPER = 0
23 SALT = 255
```

In our implementation we start by importing the necessary libraries for image manipulations such as:

- **os**: Creating directory paths and names to save image outputs.
- **numpy**: Manipulate image pixel values.
- **PIL**: Open and save images after performing processing operations.

We also define important constant values use throughout the program.

```
25
26 def build_output_file_path(output_dir, img_title, op_signature):
27     if not os.path.exists(output_dir):
28         os.makedirs(output_dir)
29
30     base_name = os.path.basename(img_title)
31     name, ext = os.path.splitext(base_name)
32     output_title = os.path.join(output_dir, f"{name}_{op_signature}{ext}")
33
34     return output_title
35
```

This function helps create the correct paths and names to save the image outputs after performing the *spatial filtering* operations. In this implementation the files are saved in the “filtered_images” directory path in the current directory where the program is located.

```
37 def get_corrupted_img(img_title):
38     """Corrupt a grayscale image by adding salt and pepper noise."""
39
40     try:
41         img = Image.open(img_title)
42         base, ext = os.path.splitext(img_title)
43         output_title = f"{base}_corrupted{ext}"
44
45         # Get the pixel data
46         img_pixels = np.array(img)
47         row, col = img_pixels.shape
48
49         # ---- Apply salt and pepper noise to the image ----
50
51         # Salt noise
52         noisy_pixels = np.random.randint(NOISY_PIXEL_LOW, NOISY_PIXEL_HIGH)
53         for _ in range(noisy_pixels):
54             x_coord = np.random.randint(0, col - 1)
55             y_coord = np.random.randint(0, row - 1)
56             img_pixels[y_coord][x_coord] = SALT
57
58         # Pepper noise
59         noisy_pixels = np.random.randint(NOISY_PIXEL_LOW, NOISY_PIXEL_HIGH)
60         for _ in range(noisy_pixels):
61             x_coord = np.random.randint(0, col - 1)
62             y_coord = np.random.randint(0, row - 1)
63             img_pixels[y_coord][x_coord] = PEPPER
64
65         # Save result
66         # Convert to uint8
67         gray_uint8 = np.clip(img_pixels, 0, 255).astype(np.uint8)
68
69         # Save the output grayscale image
70         Image.fromarray(gray_uint8).save(output_title)
71         print(f"Saved filtered image to {output_title}")
72
73         return output_title
74
75     except FileNotFoundError:
76         print(f"Error: Input image {img_title} not found.")
77     except Exception as e:
78         print("Error:", e)
79
```

This function helps us add the *salt and pepper* noise to the original gray-scale image by:

1. Opening the image file using its tile

2. Retrieve the pixel data
3. Randomly choose the amount of *salt* to be added to the image
4. Add the *salt noises* to randomly chosen pixels
5. Randomly choose the amount of *pepper* to be added to the image
6. Add the *pepper noises* to randomly chosen pixels

N.B.: When adding salt or pepper noises to a pixel value, we simply overwrite the original value by either 0 (black or *pepper noise*) or 1 (white or *salt noise*).

```

80
81 def mean_filter(img_title, filter_dim=3, output_dir="filtered_images"):
82     try:
83         if filter_dim < 1:
84             raise ValueError("filter_dim must be >= 1")
85
86         img = Image.open(img_title).convert("L") # force grayscale
87         img_pixels = np.array(img, dtype=float) # use float for accurate averaging
88         rows, cols = img_pixels.shape
89
90         # Create mean kernel
91         kernel = np.full(
92             (filter_dim, filter_dim), 1.0 / (filter_dim * filter_dim), dtype=float
93         )
94
95         # Padding size (so output size = input size)
96         pad = filter_dim // 2
97
98         # Pad using reflection to reduce border artifacts (Mirroring)
99         padded = np.pad(img_pixels, pad_width=pad, mode="reflect")
100
101         # Output array
102         out = np.zeros_like(img_pixels, dtype=float)
103
104         # Naive convolution -> O(rows * cols * filter_dim^2)
105         for r in range(rows):
106             r0 = r
107             r1 = r + filter_dim
108             for c in range(cols):
109                 c0 = c
110                 c1 = c + filter_dim
111                 window = padded[r0:r1, c0:c1]
112                 out[r, c] = np.sum(window * kernel)
113
114         output_title = build_output_file_path(
115             output_dir, img_title, f"{filter_dim}x{filter_dim}_mean"
116         )
117
118         # Clip to valid range and convert to uint8 for saving/display
119         out = np.clip(out, 0, 255).astype(np.uint8)
120         Image.fromarray(out).save(output_title)
121         print(f"Saved filtered image to {output_title}")
122
123     except FileNotFoundError:
124         print(f"Error: Input image {img_title} not found.")
125     except Exception as e:
126         print("Error:", e)
127

```

Here we implement the mean filter function which receives:

- Title of the image we want to perform the mean filter on.
- Dimension of the mean filter we want to apply on the image with default value of 3.
- Output directory to save the resulting image after applying the mean filter.

The procedure of applying the mean filter involves:

1. Create a kernel or a mask of the specified dimension using *np.full()* where each values in the kernel is equal to 1 over the square of the filter dimension because when applying the filter we want to compute the mean value of all the pixel involved, and this result will be obtained by multiplying the sum obtained by those values.
2. Using the ***mirroring*** approach to handle the edges of the image which consists using the closest pixel values in the image for the values that falls outside of the filter range.
3. Then we create a empty array of zero values to act as a *buffer* for saving the result of the convolution operation with the dimensions of the padded image.
4. Apply the filter to the image pixel values using a convolution operation where the resulting pixel values are the sum of the product of the each pixel values in the patch by the kernel which results in the mean value for the current patch.
5. Since the convolution operation has been done on floating points values, and the results might fall out of the 0-255 range which is necessary for valid picture display on a computer, we proceed to ***clip*** those values and convert them to correct ***uint8*** datatype.
6. Save the resulting array into an image for display.

```

134 def median_filter(img_title, filter_dim=3, output_dir="filtered_images"):
135     try:
136         if filter_dim < 1:
137             raise ValueError("filter_dim must be >= 1")
138         if filter_dim % 2 == 0:
139             print(
140                 f"Warning: even filter_dim -> median filter will be average of two middle values"
141             )
142
143         img = Image.open(img_title).convert("L") # force grayscale
144         img_pixels = np.array(img)
145         rows, cols = img_pixels.shape
146
147         # Padding size (so output size = input size)
148         pad = filter_dim // 2
149
150         # Pad using reflection to reduce border artifacts (Mirroring)
151         padded = np.pad(img_pixels, pad_width=pad, mode="reflect")
152
153         # Output array
154         out = np.zeros_like(img_pixels, dtype=float)
155
156         # Naive sliding-window median
157         for r in range(rows):
158             r0 = r
159             r1 = r + filter_dim
160             for c in range(cols):
161                 c0 = c
162                 c1 = c + filter_dim
163                 window = padded[r0:r1, c0:c1]
164                 out[r, c] = np.median(window)
165
166         output_title = build_output_file_path(
167             output_dir, img_title, f"{filter_dim}x{filter_dim}_median"
168         )
169
170         Image.fromarray(out.astype(np.uint8)).save(output_title)
171         print(f"Saved filtered image to {output_title}")
172
173     except FileNotFoundError:
174         print(f"Error: Input image {img_title} not found.")
175     except Exception as e:
176         print("Error:", e)
177

```

Here, we apply the median filter algorithm using a function which receives the same parameters as the previous mean filter algorithm and essentially involves:

1. Validate the filter dimension parameter passed
2. Open the input image and retrieve the pixel data in a numpy array
3. Padding the image using the ***mirroring approach*** just as in the previous mean filter function to handle the edges of the image.
4. Create an empty array to act as a buffer for the filtering operation.
5. We use the same nested loop logic as previously for applying the median filter but this time we do not compute any average value by using a kernel. Instead we pick the median value of the list of pixel values under the patch

specified by the *window* variable. This operation simply order the list of pixels values in increasing order and select the pixels value at the middle of the distribution and assign it to the pixel in the middle position of the filter or window.

6. Finally, we proceed to save the resulting pixel array as an image as *uint8* datatype. This time we don't need any *clip* operation because we were operating directly on integer values, no floating-point operation was performed, thus there is no risk of falling out of the range 0-255.

```
179 if __name__ == "__main__":  
180     corrupted_image = get_corrupted_img(INPUT_IMAGE)  
181     mean_filter(corrupted_image)  
182     mean_filter(corrupted_image, 5)  
183     median_filter(corrupted_image)  
184     median_filter(corrupted_image, 5)
```

This is the main procedure of the program where we:

1. Create a corrupted copy of the original gray-scale image by adding *salt and pepper noise* to it, and returned the saved name of the image.
2. Apply the mean filter using the default value of 3x3 in the function implementation to the corrupted image while saving the result during execution.
3. Apply a 5x5 mean filter to the same corrupted image and save the output during execution.
4. Apply the median filter using the default value of 3x3 in the function implementation to the corrupted image while saving the result during execution.
5. Apply a 5x5 median filter to the same corrupted image and save the output during execution.

The image outputs are save in the “filtered_images” directory by default and have specific names to denote which spatial filtering operation was performed on them.

2- Resulting images

Illustration 1: Original image



Illustration 2: Corrupted image



Illustration 3: 3x3 Mean Filter



Illustration 4: 3x3 Median Filter



Illustration 5: 5x5 Mean Filter



Illustration 6: 5x5 Median Filter



Illustration 7: 7x7 Mean Filter



Illustration 8: 7x7 Median Filter



Illustration 9: 9x9 Mean Filter



Illustration 10: 9x9 Median Filter



The outputs of the 3x3 median filter looks best since the edges are sharper and the smoothing effect of the median filter is not exaggerated. But for the mean filter the results tends to become even more blurrier as we increase the filter size, the one with the 3x3 filter is the best among the mean filters even though it still contains *salt and pepper noises*.